Task 1: Symmetric vs. asymmetric encryption

Symmetric encryption uses the same secret key to encrypt and decrypt data. It is fast and suitable for encrypting large amounts of information. For example, a cloud application such as Paytm can use AES encryption to protect stored transaction data.

Asymmetric encryption uses a public key and a mathematically related private key. Data encrypted with a public key can generally be decrypted only with the corresponding private key. It is commonly used in secure key exchange and digital signatures. For example, HTTPS connections used by WhatsApp Web or Paytm use asymmetric cryptography during authentication and key establishment, while symmetric encryption protects the ongoing session.

Feature	Symmetric encryption	Asymmetric encryption
Keys used	One shared secret key	Public and private key pair
Speed	Generally faster	Generally slower
Main use	Encrypting files and databases	Key exchange, authentication and signatures
Example algorithm	AES-256	RSA or elliptic-curve cryptography

Task 2: Server-side vs. client-side encryption

Server-side encryption (SSE)
Encryption happens at the cloud storage service.

Advantages
1. Easy to configure for cloud storage.

2. The service manages much of the encryption and decryption process.
3. Supports centralized key management and auditing.

Disadvantages
1. The cloud service processes decryption when authorized access is requested.
2. Access-control errors can expose decrypted data to unauthorized users.
3. Customer-managed keys add configuration and key-management responsibilities.

Client-side encryption (CSE)
The application encrypts data before uploading it.

Advantages
1. Plaintext does not need to be uploaded to the storage service.
2. The customer can retain greater control over encryption keys.
3. Helps protect data if cloud storage access is compromised but the keys remain secure.

Disadvantages
1. Key storage, recovery and rotation are more complicated.
2. Lost encryption keys may make files permanently unrecoverable.
3. Searching, previewing and processing encrypted photos becomes more difficult.

Task 3: Enable AES-256 server-side encryption on an S3 bucket

Procedure
1. Sign in to the AWS Management Console.
2. Open S3 → Create bucket.
3. Enter a globally unique bucket name, such as spotify-playlists-unique-id.
4. Choose your preferred AWS Region.
5. Keep Block all public access enabled.

6. Under the bucket's default encryption settings, choose Server-side encryption with Amazon S3 managed keys (SSE-S3).
7. Create the bucket.
8. Upload a sample text file or playlist cover image.
9. Open the object details and inspect the server-side encryption field.

SSE-S3 uses AES-256 encryption and is suitable for this exercise without requiring you to create a separate KMS key.

Task 4: Verify encryption with AWS CLI

After configuring the bucket and uploading playlist-cover.jpg, run:

```
aws s3api head-object \
  --bucket spotify-playlists-unique-id \
  --key playlist-cover.jpg
```

For an object encrypted using SSE-S3, the response should include a field similar to:

```
{
  "ServerSideEncryption": "AES256"
}
```

Task 5: Python code to upload an encrypted object

The following example uses boto3, the AWS SDK for Python, to upload a local file with SSE-S3 encryption.

```
import boto3

bucket_name = "spotify-playlists-unique-id"
file_path = "playlist-cover.jpg"
object_key = "playlist-cover.jpg"

s3 = boto3.client("s3")
```

```
s3.upload_file(
    file_path,
    bucket_name,
    object_key,
    ExtraArgs={
        "ServerSideEncryption": "AES256"
    }
)

print("File uploaded with SSE-S3 requested.")

metadata = s3.head_object(
    Bucket=bucket_name,
    Key=object_key
)

print("Encryption:", metadata.get("ServerSideEncryption"))
```